PRACTICAL - 13

Aim: Generic containers (Box.java + Driver.java)

Code Screenshot:

Box.java

```java
public class Box<T> {
    private T value;
    public void set(T value) { this.value = value; }
    public T get() { return value; }
}

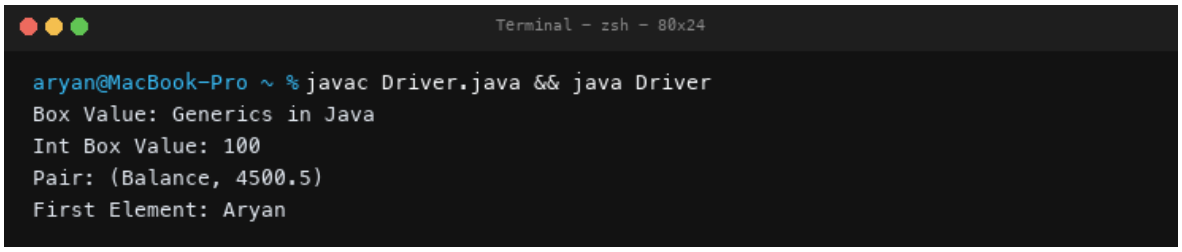
class Pair<A, B> {
    private A first; private B second;
    public Pair(A a, B b) { this.first = a; this.second = b; }
    public String toString() { return "(" + first + ", " + second + ")"; }
}
```

Driver.java

```java
public class Driver {
    public static <T> T getFirst(T[] arr) {
        return (arr != null && arr.length > 0) ? arr[0] : null;
    }

    public static void main(String[] args) {
        Box<String> strBox = new Box<>();
        strBox.set("Generics in Java");
        Box<Integer> intBox = new Box<>();
        intBox.set(100);
        Pair<String, Double> pair = new Pair<>("Balance", 4500.50);
        String[] names = { "Aryan", "Karan", "Rohan" };
        System.out.println("Box Value: " + strBox.get());
        System.out.println("Int Box Value: " + intBox.get());
        System.out.println("Pair: " + pair);
        System.out.println("First Element: " + getFirst(names));
    }
}
```

Output Screenshot:



Learning Outcome:

- Designed custom type-parameterized generic classes (Box).
- Handled multiple independent type variables (Pair).
- Implemented generic methods with independent type parameters (T getFirst).
- Enforced compile-time type safety without runtime casting.
- Understood how generics eliminate ClassCastException bugs.

Aim: Bounded stats (StatsCalculator.java + Driver.java)

Code Screenshot:

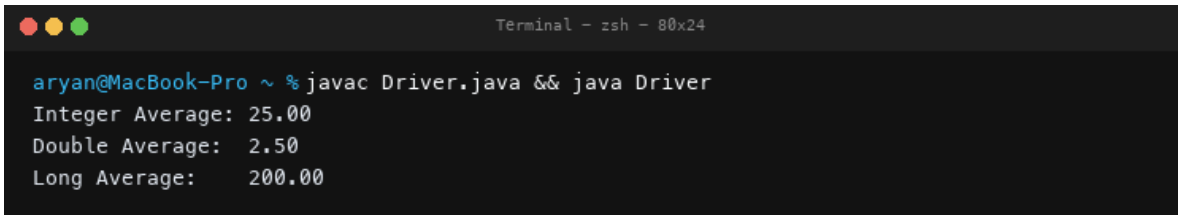
StatsCalculator.java

```java
public class StatsCalculator {
    public static <N extends Number> double average(N[] values) {
        if (values == null || values.length == 0) return 0.0;
        double sum = 0.0;
        for (N v : values) sum += v.doubleValue();
        return sum / values.length;
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Integer[] intArr = { 10, 20, 30, 40 };
        Double[] dblArr = { 1.5, 2.5, 3.5 };
        Long[] lngArr = { 100L, 200L, 300L };
        System.out.printf("Integer Average: %.2f\n", StatsCalculator.average(intArr));
        System.out.printf("Double Average:  %.2f\n", StatsCalculator.average(dblArr));
        System.out.printf("Long Average:    %.2f\n", StatsCalculator.average(lngArr));
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered upper-bounded type parameters using .
- Accessed common superclass methods (doubleValue()) directly on generic types.
- Enforced compile-time restrictions preventing invalid non-numerical arguments.
- Processed diverse wrapper types under a single algorithm.
- Deepened appreciation for Java's strong generic type system.